Project Requirements – Group 7

James Markijohn and Samaa Hediya

Overview

The web application being developed is an art gallery management tool, which can track individuals (guests and employees) who enter and exit the various rooms of the art gallery, as well as the art gallery itself. The tool will also be used to track user events, which will be stored in a database that may be queried by administrators and employees. Guest users must create an account when they enter the gallery and will be able to use the web application as a map, but will not be able to see the locations of any other guests or employees. Employees will be able to see the locations of guests and employees and will be able to query the movement event database. They may also add movement events to the database but may not delete them. Administrators (a subset of employees) will be able to see all users in the gallery and will be able to query the full database, as well as add or delete events in all tables of the database except the security log.

The web application will be accessed through a login page that will be publicly exposed. Depending on the role of the user who logs in, different functionality will be authorized for that session. For instance, when a guest logs in, they will have no option to access the event database, whereas an administrator will be authorized to do so.

Event logging will be implemented with respect to the current state of a user in the gallery. That is, a user may only be able to occupy one room at once, may not leave a room or the gallery if they are not in that room or the gallery, and should not be able to enter a non-adjacent room.

This application will make use of the ReactJS library for its frontend and UI elements, Django (particularly Django REST Framework) for its backend and API alongside Python, and a relational database using PostgreSQL. Caddy will be used for reverse proxying and HTTPS support. The PostgreSQL database will be accessible via an API exposed by the backend, which will allow privileged users to perform certain operations on the database. This web application is containerized with Docker, and will consist of database, backend, and frontend reverse proxy containers. The database container will be isolated from the internet, while the public reverse proxy will be accessible and will serve the frontend; the backend is not reachable from the outside.

Functional Security Requirements

The following functional security requirements were identified. Each requirement category is listed below, with the requirements described in their respective category.

1. *Authentication.* Users will authenticate into this application using an email and password combination (something they know) followed by a multi-factor authentication method (TOTP, something they have). Additionally, if a user attempts to log in but supplies incorrect credentials five times in a row, they will be locked out of their account for five minutes; this will prevent brute-force password attacks. Guest users will be permitted to delete their account, but when this option is selected, they will be forced to reauthenticate (both password and MFA) before the deletion is processed. This will not delete associated movement event data in that table of the database, or any security events that were associated with that account. Passwords can be reset by users, but only if they can successfully authenticate using their MFA method. MFA methods may only be updated by administrators.
2. *Authorization*. When a user authenticates to this application, they will be authorized to access certain elements based on their roles. A guest user will be able to access a map of the gallery, see only their location in the gallery, and perform functions such as password resets or account deletions on only their accounts. Employees can do everything a guest can but can also query the movement event database and see the location of all users in the gallery. Administrators will be authorized to have the same level of access as employees and can also modify any accounts; they may also query and modify all tables in the database aside from the security log. This is so that any bad or duplicate data in the movement event table can be deleted, in the event such data is entered by the application. Roles will be enforced at every endpoint of this system, and access will be denied by default unless a user attempting to access a resource has the specific role required.
3. *Data Protection.* Data will be protected through encryption, both in-transit and at-rest. HTTPS will be used for this web application to encrypt the data while it is in-transit, while field-level encryption will be used to protect sensitive data in the database. Field-level encryption will use a secret key and a secure encryption library to encrypt and decrypt fields that need protection, such as the TOTP secret and any other associated MFA information, the full names of users, an optional phone number that users may provide, and the security log fields. Additionally, passwords will be salted and stored as hashes using the Argon2id algorithm (via Django's password hashers), not in plaintext. Personal information associated with accounts will be visible to all employees and administrators. However, no guest may view another guest's personal information (this includes full names and contact information). Secrets are managed with a .env file, which is never committed to any repository, and no default credentials are used.
4. *Input Validation and Output Encoding*. Users will be able to input data to this web application on the login and account creation or deletion pages, as well as on the database

query page (which will feature structured filters and will be accessible to employees and administrators only). At the account creation and login pages, the email field will check to ensure the input conforms to a valid email syntax. Fields where users supply their name will include a character limit. This validation will be performed server-side so that the system is not relying on the integrity of a client's browser. Additionally, SQL injections will be prevented with parameterized queries, which will protect the database. In terms of output, any output from the application will be encoded such that it cannot be interpreted as code by the client that made the request. For instance, if an attacker were to add malicious code to the database, and a user attempts to log in and receives that malicious code as output from the web app, the output will be encoded with escape characters so that the code cannot be executed.

5. *Monitoring and Logging*. Security events will be logged in a separate table of the database, which will only be accessible to administrators and cannot be modified by any users due to restricted database permissions. These logs will include information about who made database queries and for what data, account deletions and creations, account modifications, authentication successes and failures, and other noteworthy events. All fields in this table, along with sensitive fields throughout the rest of the database, will be encrypted to preserve its confidentiality. Logs may then be monitored by administrators who are investigating potential security incidents.
6. *DDoS Protection and Availability*. To avoid DDoS attacks, rate limits will be employed at both the login pages (for instance, no more than ten login attempts in five minutes from one client before that client is banned for ten minutes) and the database query pages (for example, 10 queries per account per minute). These protections will prevent the web application from being overwhelmed with requests, which will preserve its availability. It is worth acknowledging that these rate limits have the potential to be abused and prevent legitimate users from accessing the application. A backup solution will also be implemented so that corruption or loss of data will not be catastrophic, specifically through scheduled encrypted database dumps.
7. *Cross-Site Attacks*. To avoid the common cross-site scripting and cross-site request forgery attacks, mitigation techniques for both will be used. HTML encoding and sanitization will be used to prevent cross-site scripting, and a CSRF token will be used in accordance with OWASP's recommendations to avoid cross-site request forgeries. Django's CSRF protection will be used when building this application. XSS attacks will be prevented with React's automatic escaping features, along with avoiding the use of its dangerouslySetInnerHTML element.
8. *Session Security.* Django's session framework will only store sessions server-side, and a client's browser will only hold a cryptographically secure randomized session ID in a cookie. This prevents session data from being stored in a token on the client device. Secure cookie attributes such as HttpOnly (no JavaScript reading of cookies), Secure (enforcing HTTPS for cookies) and SameSite with the Strict limit enforced (for CSRF

protection) will be set. Sessions will end after eight hours of inactivity, which will trigger the destruction of the session on the server (as will a logout event). A new session ID will be issued upon a successful login. Sessions will also be revoked following a security event (such as a password reset). When a user has supplied the correct email and password but has not passed MFA, the session will end after one minute of inactivity. Concurrent sessions will be permitted.

9. *Error Handling.* Errors will be provided when appropriate to maintain the usability of this system. However, these errors will be as vague and generic as possible without becoming completely useless to a user. This will help prevent unintentional information disclosure. An example of this will be a message saying “Incorrect email address or password” when a user fails to provide valid credentials; note that the incorrect credential is not specified.
10. *Race Conditions and Data Duplication.* Race conditions will be mitigated by ensuring that event validation and state updates occur in a single transaction. Data may be deleted by administrators if duplicate records are created. If a user leaves the gallery, and the movement event indicating that they left the gallery is subsequently deleted, they will still be recognized as being outside of the gallery. This is due to a flag that will be included in the accounts table of the database, indicating if a user is in the gallery.
11. *Security Misconfigurations.* Care will be taken to ensure that unnecessary stack traces will never be presented to the user, and that any software with a debug mode used in this application will have that mode disabled. Ports will only be exposed when absolutely necessary for the sake of network security.

Non-Functional Requirements

In addition to the security requirements that were identified above, there are several non-functional requirements that will be emphasized during development. These are listed and explained below.

Performance and Scalability. The web application will be designed with performance in mind, specifically when under load. For instance, 100 users should expect to use this application without experiencing more than one second of delay between submitting a request and receiving a response. If too many requests are sent to the application, users will be provided with errors stating this, and the application will reject the request rather than attempting to queue it or skip other requests to process it. The rate limits mentioned previously will assist with this as well, insofar as DDoS attacks will have a more difficult time affecting the performance of the application. Scalability is largely beyond the scope of this project, but since the application is containerized, it is entirely possible for more containers to be added behind the Caddy reverse proxy to support a larger workload.

Availability. The web application's availability matters from both security and functionality standpoints. Measures to keep the application available have already been discussed (such as making use of network security features and taking backups), but they are worth repeating in this section. High availability will lead to a better user experience for the guests of the art gallery and will ensure that all movement is properly tracked.

Usability. The user interface of this web application will be designed with usability in mind, especially for interfaces that are exposed to guests of the gallery. For example, since MFA is being implemented in this web app, the prompts and setup process will include clear instructions and a simple interface. The user interface will be simple to navigate and will feature familiar design patterns. From the perspective of the employees and administrators, the four database tables in this system (movement events, security logs, accounts, and gallery rooms) will provide convenient locations to store and lookup data pertaining to the gallery, such as information associated with each room, or logs for any significant security events.

Security Goals and Objectives

The following security goals and objectives have been identified. While more may be identified in time, these are the fundamental objectives that will be considered during development.

Maintaining CIA. Confidentiality, integrity, and availability will be maintained during this web application's operations. Data will be encrypted in-transit (HTTPS using TLS) and at-rest (using field-level encryption) to preserve confidentiality. Integrity will be provided by TLS (which will be capable of detecting tampering while data is in-transit), constraints on who can edit which tables in the database, and the append-only security log which may never be modified by any user with any role. Availability will be protected by the DDoS rate limits described previously, which will prevent attackers from completely overwhelming the application. Backups will also be maintained so that data can quickly be restored in the event of data corruption or loss.

Auditability of User Actions. The security log table will store all notable events that occur (see the *Monitoring and Logging* section of the Functional Security Requirements page for details), which will also provide non-repudiation of user behavior. Administrators will be able to identify which user account is responsible for which actions by querying the security log. The web app will be designed so that users cannot bypass the logging mechanisms that are used to detect and write these events to the database. Additionally, the security log table's fields will be encrypted.

Protection of Location Data. Since this web application will be monitoring users as they move through the art gallery, the locations of guest users should be protected. Only employees and administrators will be allowed to see the locations of all guest users, while guest users should only be able to see their own location in the gallery. Employees and administrators are given full access to location information in case they need to locate one of their colleagues, or if they need to provide support to one of the guests. Since the movement event table of the database will log every time a user moves from one room to another, access to this table is also restricted to employees and administrators only. This ensures that no guests can access and examine historical location data for other guest users.

Compliance and Legal Considerations

This application will track movement of users in a location and will store some of their contact details and personal information. Since PII is stored in our databases, regulations regarding data privacy were reviewed and applied to this application's design. The majority of this section is based on the GDPR's requirements. PCI DSS and HIPAA were not found to be applicable to this system.

Data Collection. In accordance with the GDPR, no unnecessary information on users will be collected. Users may provide an additional contact method (a phone number) on their own volition; this is also the case for their full name. Once a user leaves the art gallery premises, they will no longer be tracked, and no data will be added to the movement event database. Since employees and guests are monitored by this application, an art gallery using this system may wish to alert employees and guests of the electronic monitoring occurring in this case.

Breaches. In the event of a data breach, administrators would be able to use the security log to investigate noteworthy events, correlate data between the other tables of the database, and prepare a statement for the users of this application and regulatory entities within a timely manner (the GDPR stipulates 72 hours for the latter).

Data Processing and Retention. Data is not being used by this application for any purposes other than tracking user movements, logging events that occur within the gallery and the application, and retaining account information. This data is being encrypted in-transit, and the most sensitive fields (specified previously) are being encrypted at-rest. Data is currently retained indefinitely by the application.

Privacy. Users are permitted to delete their accounts at any time and do not need to specify a reason, aligning with the GDPR's right to erasure principles. This process will pseudonymize their information in the accounts table of the database while ensuring that movement or security events can still be correlated to that account. This balances the needs of the application administrators (investigating security incidents involving a deleted account) with the GDPR's privacy-respecting stipulations (identifying details are removed from the accounts table).